

codigo python pronto para a simulação

"""

UNIVERSIDADE DE ENGENHARIA

DEPARTAMENTO DE ENGENHARIA DE ENERGIA / PROCESSOS

PROJETO: SISTEMA DE ANÁLISE DE EFICIÊNCIA E ESTABILIDADE EM SISTEMAS DE DISTRIBUIÇÃO DE GÁS NATURAL

AUTOR: Desenvolvedor Acadêmico / Engenharia

DATA: 2026

Descrição: Ferramenta computacional para monitoramento, análise de risco e simulação dinâmica de redes de distribuição de gás natural.

"""

import os

import random

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

Configuração para exibir todas as colunas do Pandas no terminal de forma legível

pd.set_option('display.max_columns', None)

pd.set_option('display.width', 1000)

#

=====

=====

1. FUNÇÕES DE INICIALIZAÇÃO E GERAÇÃO DE CENÁRIOS

#

=====

=====

def gerar_dados_bairros():

"""

Gera automaticamente dados sintéticos realistas para 10 bairros de uma cidade, garantindo a coerência física e operacional de uma rede de gás natural.

"""

random.seed(42) # Fixa semente para reprodutibilidade acadêmica

np.random.seed(42)

nomes_bairros = [

"Centro Histórico", "Jardim América", "Vila Industrial", "Bairro das Nações",

"Zona Sul Residenciais", "Parque Tecnológico", "Alto da Colina",

"Distrito Portuário", "Complexo Comercial", "Nova Aurora"

]

dados = []

for bairro in nomes_bairros:

```

# Demanda diária base em m³/dia
demanda = float(np.random.randint(5000, 25000))
# Capacidade máxima da estação reguladora (m³): margem de segurança sobre a
demanda
capacidade_maxima = float(demanda * np.random.uniform(1.5, 2.5))
# Volume inicial disponível (m³): tanque ou pulmão operacional da estação
volume_inicial = float(demanda * np.random.uniform(1.0, 5.5))
# Percentual de perda física na tubulação (%)
percentual_perda = float(np.random.uniform(1.5, 8.0))
# Pressão média operacional da sub-rede (bar) - Alvo de projeto = 5.0 bar
pressao_media = float(np.random.uniform(2.8, 5.8))
# Custo de distribuição por metro cúbico (R$/m³)
custo_por_m3 = float(np.random.uniform(0.12, 0.28))

dados.append({
    "Bairro": bairro,
    "Demanda_Base": demanda,
    "Capacidade_Max": capacidade_maxima,
    "Volume_Inicial": volume_inicial,
    "Percentual_Perda": percentual_perda,
    "Pressao_Media_Base": pressao_media,
    "Custo_M3": custo_por_m3
})

```

```

return pd.DataFrame(dados)

```

```

#
=====
=====
# 2. FUNÇÕES DE PROCESSAMENTO MATEMÁTICO E ENGENHARIA
#
=====
=====

```

```

def calcular_indicadores_operacionais(df):

```

```

    """

```

```

    Aplica as equações de engenharia para determinar os volumes, eficiências,
    estabilidade de pressão, custos e a classificação de risco das sub-redes.
    Utiliza operações vetorizadas com NumPy para maior eficiência computacional.
    """

```

```

# 1. Volume Perdido (m³)

```

```

df["Volume_Perdido"] = df["Demanda_Base"] * (df["Percentual_Perda"] / 100.0)

```

```

# 2. Volume Efetivamente Entregue (m³)

```

```

df["Volume_Entregue"] = df["Demanda_Base"] - df["Volume_Perdido"]

```

```

# 3. Eficiência da Distribuição (%)

```

```

df["Eficiencia"] = (df["Volume_Entregue"] / df["Demanda_Base"]) * 100.0

```

4. Custo Operacional Diário (R\$)

```
df["Custo_Operacional"] = df["Volume_Entregue"] * df["Custo_M3"]
```

5. Autonomia da Estação Reguladora (Dias)

```
df["Autonomia_Dias"] = df["Volume_Inicial"] / df["Demanda_Base"]
```

6. Índice de Estabilidade da Pressão (IEP) - Pressão Ideal = 5.0 bar

```
df["IEP"] = df["Pressao_Media_Base"] / 5.0
```

7. Classificação de Risco Operacional Automatizada

```
condicoes = [
```

```
    (df["Autonomia_Dias"] > 4.0),
```

```
    (df["Autonomia_Dias"] >= 2.0) & (df["Autonomia_Dias"] <= 4.0),
```

```
    (df["Autonomia_Dias"] < 2.0)
```

```
]
```

```
escolhas = ["BAIXO", "MÉDIO", "ALTO"]
```

```
df["Classificacao_Risco"] = np.select(condicoes, escolhas, default="N/A")
```

```
return df
```

```
def monitorar_alertas_instantaneos(df):
```

```
    """
```

```
    Garante o monitoramento de segurança operacional emitindo alertas visuais imediatos  
    no terminal caso algum parâmetro de engenharia fuja dos limites seguros.
```

```
    """
```

```
    print("\n" + "="*80)
```

```
    print(" SISTEMA DE MONITORAMENTO EM TEMPO REAL - ALERTAS CRÍTICOS  
    DETECTADOS")
```

```
    print("="*80)
```

```
    alertas_disparados = 0
```

```
    for _, linha in df.iterrows():
```

```
        bairro = linha["Bairro"]
```

```
        if linha["Autonomia_Dias"] < 1.0:
```

```
            print(f"[ALERTA CRÍTICO - ABASTECIMENTO] Bairro: {bairro} | Autonomia inferior a  
1 dia ({linha['Autonomia_Dias']:.2f} dias)!")
```

```
            alertas_disparados += 1
```

```
        if linha["Eficiencia"] < 75.0:
```

```
            print(f"[ALERTA CRÍTICO - EFICIÊNCIA] Bairro: {bairro} | Perdas elevadas!  
Eficiência em {linha['Eficiencia']:.2f}%!")
```

```
            alertas_disparados += 1
```

```
        if linha["Pressao_Media_Base"] < 3.0:
```

```
            print(f"[ALERTA CRÍTICO - PRESSÃO] Bairro: {bairro} | Subpressão na rede!  
Pressão em {linha['Pressao_Media_Base']:.2f} bar (Risco de retrocesso de chama)!")
```

```

    alertas_disparados += 1

if alertas_disparados == 0:
    print("[INFO] Todos os parâmetros operacionais instantâneos operam dentro dos
limites seguros estabelecidos.")
    print("*80)

#
=====
=====
# 3. MÓDULO DE SIMULAÇÃO OPERACIONAL DINÂMICA (10 DIAS)
#
=====
=====

def executar_simulacao_temporal(df_inicial, dias=10):
    """
    Simula dinamicamente a operação de malha de distribuição de gás por N dias.
    Modula a demanda estocasticamente em +/-15%, flutua a pressão da rede de transporte,
    atualiza o balanço de massa diário dos reservatórios e registra a evolução temporal.
    """
    # Inicialização de matrizes/dicionários dinâmicos de histórico temporal
    historico_volumes = {bairro: [] for bairro in df_inicial["Bairro"]}
    historico_pressoes = {bairro: [] for bairro in df_inicial["Bairro"]}
    historico_eficiencia_sistema = []
    dias_criticos = []

    # Cópia de trabalho dos volumes disponíveis nas estações
    volumes_atuais = df_inicial.set_index("Bairro")["Volume_Inicial"].to_dict()

    print("\nIniciando Simulação Operacional Avançada Estocástica (10 Dias
Consecutivos)...")

    for dia in range(1, dias + 1):
        total_entregue_dia = 0
        total_demanda_dia = 0
        alertas_dia = 0

        for idx, linha in df_inicial.iterrows():
            bairro = linha["Bairro"]

            # Flutuação estocástica da demanda (+/- 15%) usando distribuição uniforme
            fator_demanda = random.uniform(0.85, 1.15)
            demanda_simulada = linha["Demanda_Base"] * fator_demanda

            # Flutuação transiente da pressão operacional
            fator_pressao = random.uniform(0.75, 1.20)
            pressao_simulada = max(1.0, linha["Pressao_Media_Base"] * fator_pressao)

```

```

# Balanço de Massa e cálculo de perdas
perda_simulada = demanda_simulada * (linha["Percentual_Perda"] / 100.0)
entregue_simulado = demanda_simulada - perda_simulada

# Atualização dos tanques/pulmões operacionais do bairro
volumes_atuais[bairro] = max(0.0, volumes_atuais[bairro] - entregue_simulado)

# Registros históricos diários
historico_volumes[bairro].append(volumes_atuais[bairro])
historico_pressoes[bairro].append(pressao_simulada)

total_entregue_dia += entregue_simulado
total_demanda_dia += demanda_simulada

# Avaliação de criticidade diária do nó da rede
if volumes_atuais[bairro] / linha["Demanda_Base"] < 1.0 or pressao_simulada < 3.0:
    alertas_dia += 1

# Eficiência ponderada de toda a rede integrada no dia
eficiencia_sistema_dia = (total_entregue_dia / total_demanda_dia) * 100.0
historico_eficiencia_sistema.append(eficiencia_sistema_dia)

# Classificação de severidade do dia operacional
if alertas_dia >= 3 or eficiencia_sistema_dia < 93.0:
    dias_criticos.append(dia)

print(f"Simulação concluída com sucesso. Dias críticos mapeados no período:
{dias_criticos}")
return historico_volumes, historico_pressoes, historico_eficiencia_sistema, dias_criticos

#
=====
=====
# 4. ENGINE DE EXIBIÇÃO DE RELATÓRIOS E COMPILADOR DE INDICADORES
#
=====
=====

def exibir_tabela_formatada(df):
    """
    Formata e exibe os dados operacionais estruturados como uma tabela de dados
    de engenharia, facilitando a inspeção rápida de valores no terminal.
    """
    print("\n" + "="*115)
    print("          RELATÓRIO MATRICIAL COMPLETO DE OPERAÇÃO DO SISTEMA
DE GÁS")
    print("="*115)

```

```

print(f'{"Bairro":<22} | {"Demanda (m³)":<12} | {"Perda (m³)":<10} | {"Entregue (m³)":<13} | {"Efic. (%)":<9} | {"Custo (R$)":<10} | {"Auton.(d)":<8} | {"IEP":<5} | {"Risco":<6}')
print("-"*115)

```

```

for _, reg in df.iterrows():
    print(f'{reg["Bairro"]:<22} | '
          f'{reg["Demanda_Base"]:<12.1f} | '
          f'{reg["Volume_Perdido"]:<10.1f} | '
          f'{reg["Volume_Entregue"]:<13.1f} | '
          f'{reg["Eficiencia"]:<9.2f} | '
          f'{reg["Custo_Operacional"]:<10.2f} | '
          f'{reg["Autonomia_Dias"]:<8.2f} | '
          f'{reg["IEP"]:<5.2f} | '
          f'{reg["Classificacao_Risco"]:<6}')
print("-"*115)

```

```

def gerar_analise_diagnostica_final(df, dias_criticos, historico_eficiencia):

```

```

    """
    Analisa os dados compilados no Pandas e gera um relatório textual analítico estruturado
    contendo diagnósticos estatísticos fundamentados para a tomada de decisão da
    gerência.
    """

```

```

    # Identificações automáticas de extremos

```

```

    bairro_mais_critico = df.loc[df["Autonomia_Dias"].idxmin(), "Bairro"]
    bairro_mais_eficiente = df.loc[df["Eficiencia"].idxmax(), "Bairro"]
    maior_custo_bairro = df.loc[df["Custo_Operacional"].idxmax(), "Bairro"]
    menor_estabilidade_pressao = df.loc[df["IEP"].idxmin(), "Bairro"]

```

```

    # Métricas agregadas do sistema completo

```

```

    media_eficiencia = df["Eficiencia"].mean()
    media_perdas = df["Volume_Perdido"].mean()
    media_pressao = df["Pressao_Media_Base"].mean()
    custo_total_rede = df["Custo_Operacional"].sum()
    quantidade_alto_risco = int((df["Classificacao_Risco"] == "ALTO").sum())

```

```

    print("\n" + "#"*80)

```

```

    print("          RELATÓRIO FINAL INTEGRADO DE ENGENHARIA — SUMÁRIO
EXECUTIVO")

```

```

    print("#"*80)

```

```

    print(f'-> Média Geral de Eficiência das Sub-redes: {media_eficiencia:.2f}%')
    print(f'-> Média de Perdas por Distribuição de Nó: {media_perdas:.2f} m³/dia')
    print(f'-> Pressão Média Hidrostática da Rede Geral: {media_pressao:.2f} bar')
    print(f'-> Custo Operacional Total de Escalonamento: R$ {custo_total_rede:.2f}/dia')
    print(f'-> Quantidade de Setores em Risco Alto: {quantidade_alto_risco} setor(es)')
    print("-" * 80)

```

```

    print(" ANÁLISE COMPLEMENTAR DE EXTREMOS E VULNERABILIDADES DE
REDE:")

```

```

    print(f" * Nó Mais Crítico (Menor Autonomia de Suprimento):    {bairro_mais_critico}")
    print(f" * Nó Mais Eficiente (Menor Coeficiente de Fricção/Perda):
{bairro_mais_eficiente}")
    print(f" * Gargalo de Custos Operacionais (Maior Consumo/Gasto):
{maior_custo_bairro}")
    print(f" * Setor com Menor Estabilidade de Pressão Dinâmica:
{menor_estabilidade_pressao}")
    print("-" * 80)
    print(" CONCLUSÕES COMPUTAÇÃO-SISTÊMICAS AUTOMATIZADAS:")

    if quantidade_alto_risco > 0:
        print(" [PARECER] RECOMENDAÇÃO DE INTERVENÇÃO IMEDIATA. Existem
setores operando abaixo da margem de")
        print("      contingência de 2 dias de autonomia. Risco iminente de despressurização
localizada por pico de demanda.")
    else:
        print(" [PARECER] O sistema apresenta estabilidade de suprimento adequada em
curto prazo. Todos os setores")
        print("      possuem volume regulador acima do limite operacional de 2 dias.")

    if media_eficiencia < 95.0:
        print(" [ALERTA] O índice de perda física geral está acima do padrão de excelência
internacional (95%).")
        print("      Recomenda-se varredura acústica com detectores de vazamento nas
sub-redes identificadas.")
    else:
        print(" [SUCESSO] Excelente eficiência de transporte. Integridade estrutural das
tubulações em conformidade.")

    print(f" [SIMULAÇÃO] Dias Críticos identificados durante o ciclo transiente de 10 dias:
{dias_criticos}")
    print(f"      Eficiência Média da Rede ao término da Simulação:
{np.mean(historico_eficiencia):.2f}%")
    print("#"*80 + "\n")

#
=====
=====
# 5. MÓDULO DE GERAÇÃO E EXPORTAÇÃO DE GRÁFICOS (MATPLOTLIB)
#
=====
=====

def plotar_e_salvar_graficos(df, hist_vol, hist_pres, hist_efic):
    """
    Gera as 7 representações visuais obrigatórias utilizando técnicas de plotagem
    profissionais em Matplotlib, salvando-as como arquivos PNG.
    """

```

```

print("Gerando e salvando representações gráficas do sistema...")
bairros = df["Bairro"].values

# -----
# FIGURA 1: Gráficos Operacionais Estáticos Estáveis (4 Gráficos em um Grid
Multi-Painel)
# -----
fig, axs = plt.subplots(2, 2, figsize=(16, 11))
fig.suptitle("SISTEMA DE DISTRIBUIÇÃO DE GÁS NATURAL - INDICADORES DE
REDE", fontsize=16, fontweight='bold', y=0.98)

# 1. Gráfico de Barras: Demanda Diária vs Volume Entregue
x = np.arange(len(bairros))
largura = 0.35
axs[0, 0].bar(x - largura/2, df["Demanda_Base"], largura, label='Demanda Requerida',
color='#3182bd')
axs[0, 0].bar(x + largura/2, df["Volume_Entregue"], largura, label='Volume Entregue',
color='#6baed6')
axs[0, 0].set_title('Balanço de Volume: Solicitado vs Entregue por Bairro', fontsize=12,
fontweight='bold')
axs[0, 0].set_ylabel('Volume (m³/dia)', fontsize=10)
axs[0, 0].set_xticks(x)
axs[0, 0].set_xticklabels(bairros, rotation=35, ha='right', fontsize=8)
axs[0, 0].legend()
axs[0, 0].grid(True, linestyle='--', alpha=0.5)

# 2. Gráfico de Barras: Custo Operacional por Bairro
axs[0, 1].bar(bairros, df["Custo_Operacional"], color='#e6550d', edgecolor='black',
alpha=0.8)
axs[0, 1].set_title('Custo Operacional Diário por Sub-Rede', fontsize=12, fontweight='bold')
axs[0, 1].set_ylabel('Custo (R$/dia)', fontsize=10)
axs[0, 1].set_xticklabels(bairros, rotation=35, ha='right', fontsize=8)
axs[0, 1].grid(True, linestyle='--', alpha=0.5)
# Adicionando rótulos de valores nas barras
for idx, valor in enumerate(df["Custo_Operacional"]):
    axs[0, 1].text(idx, valor + (valor*0.01), f"R${int(valor)}", ha='center', va='bottom',
    fontsize=7, fontweight='bold')

# 3. Gráfico de Linha: Eficiência de Distribuição por Bairro
axs[1, 0].plot(bairros, df["Eficiencia"], marker='o', color='#2ca02c', linewidth=2,
markersize=7)
axs[1, 0].axhline(75.0, color='red', linestyle=':', label='Limite Alerta Crítico (75%)')
axs[1, 0].set_title('Eficiência de Escoamento Hidráulico por Nó', fontsize=12,
fontweight='bold')
axs[1, 0].set_ylabel('Eficiência (%)', fontsize=10)
axs[1, 0].set_ylim(min(df["Eficiencia"])*0.95, 101)
axs[1, 0].set_xticklabels(bairros, rotation=35, ha='right', fontsize=8)
axs[1, 0].legend()

```



```

    axs[1, 0].grid(True, linestyle='--', alpha=0.6)

# 4. Gráfico de Pizza: Níveis de Risco Operacional Geral
contagem_risco = df["Classificacao_Risco"].value_counts()
cores_risco = {'BAIXO': '#2ca02c', 'MÉDIO': '#ff7f0e', 'ALTO': '#d62728'}
cores_plot = [cores_risco[risco] for risco in contagem_risco.index]

    axs[1, 1].pie(contagem_risco, labels=contagem_risco.index, autopct='%1.1f%%',
        startangle=140, colors=cores_plot, wedgeprops={'edgecolor': 'white', 'linewidth':
1.5})
    axs[1, 1].set_title('Distribuição Consolidada de Risco Operacional', fontsize=12,
fontweight='bold')

    plt.tight_layout()
    plt.savefig("indicadores_estaticos_sistema.png", dpi=300)
    plt.close()

# -----
# FIGURA 2: Gráficos de Evolução Dinâmica e Simulação (3 Linhas Temporais
Complementares)
# -----
fig_sim, axs_sim = plt.subplots(3, 1, figsize=(14, 12), sharex=False)
fig_sim.suptitle("ANÁLISE DE COMPORTAMENTO DINÂMICO TRANSIENTE —
SIMULAÇÃO 10 DIAS", fontsize=15, fontweight='bold')

    dias_eixo = np.arange(1, 11)

# 5. Gráfico de Linha Temporal: Evolução do Volume Disponível
for bairro in hist_vol.keys():
    axs_sim[0].plot(dias_eixo, hist_vol[bairro], marker='s', linestyle='-', label=bairro,
alpha=0.7)
    axs_sim[0].set_title("Evolução dos Volumes Disponíveis nos Pulmões Reguladores",
fontsize=11, fontweight='bold')
    axs_sim[0].set_ylabel("Volume (m³)", fontsize=9)
    axs_sim[0].grid(True, linestyle=':', alpha=0.6)
    axs_sim[0].set_xticks(dias_eixo)
    axs_sim[0].legend(loc='center left', bbox_to_anchor=(1, 0.5), fontsize=8)

# 6. Gráfico de Linha Temporal: Variação da Pressão
for bairro in hist_pres.keys():
    axs_sim[1].plot(dias_eixo, hist_pres[bairro], marker='^', linestyle='--', alpha=0.6)
    axs_sim[1].axhline(3.0, color='red', linestyle='-', label='Pressão de Alerta Mínimo (3 bar)')
    axs_sim[1].set_title("Variação da Pressão Operacional Dinâmica por Sub-Rede",
fontsize=11, fontweight='bold')
    axs_sim[1].set_ylabel("Pressão (bar)", fontsize=9)
    axs_sim[1].grid(True, linestyle=':', alpha=0.6)
    axs_sim[1].set_xticks(dias_eixo)

```

```

# 7. Gráfico de Linha Temporal: Eficiência Média Diária do Sistema Integrado
axs_sim[2].plot(dias_eixo, hist_efic, color='#7f7f7f', marker='o', linewidth=3, markersize=8,
label='Eficiência da Malha')
axs_sim[2].set_title("Evolução da Eficiência Média Diária Consolidada do Sistema",
fontsize=11, fontweight='bold')
axs_sim[2].set_xlabel("Dia da Simulação", fontsize=10)
axs_sim[2].set_ylabel("Eficiência (%)", fontsize=9)
axs_sim[2].grid(True, linestyle='-', alpha=0.4)
axs_sim[2].set_xticks(dias_eixo)
axs_sim[2].set_ylim(85, 100)
for idx, val in enumerate(hist_efic):
    axs_sim[2].text(dias_eixo[idx], val + 0.5, f"{val:.1f}%", ha='center', fontsize=8,
color='black', fontweight='bold')

```

```

plt.tight_layout()
plt.savefig("simulacao_dinamica_10dias.png", dpi=300)
plt.close()
print("[SUCESSO] Gráficos exportados com qualidade de publicação científica:")
print("      1. 'indicadores_estaticos_sistema.png'")
print("      2. 'simulacao_dinamica_10dias.png'")

```

```

#
=====
=====
# 6. MENU INTERATIVO E ORQUESTRADOR CENTRAL DO SISTEMA
#
=====
=====

```

```

def main():
    """
    Controlador central responsável por encadear a geração dos cenários, computar os
    indicadores estruturados, disparar a simulação e exportar os relatórios acadêmicos.
    """

    print("="*80)
    print(" SISTEMA DE ANÁLISE DE EFICIÊNCIA E ESTABILIDADE EM REDES DE GÁS
    NATURAL v1.0")
    print("="*80)

    # Passo 1: Geração Autônoma e Consistente de Dados
    df_rede = gerar_dados_bairros()

    # Passo 2: Processamento Computacional Matemático Base (Equações de Engenharia)
    df_rede = calcular_indicadores_operacionais(df_rede)

    # Passo 3: Execução do Sistema de Alertas em Tempo Real
    monitorar_alertas_instantaneos(df_rede)

```

```
# Passo 4: Exibição Estruturada de Resultados Estáticos
exibir_tabela_formatada(df_rede)
```

```
# Passo 5: Execução Automática da Simulação Transiente Estocástica de 10 dias
hist_vol, hist_pres, hist_efic, dias_criticos = executar_simulacao_temporal(df_rede,
dias=10)
```

```
# Passo 6: Geração de Diagnósticos Automatizados Finais
gerar_analise_diagnostica_final(df_rede, dias_criticos, hist_efic)
```

```
# Passo 7: Plotagem Automática e Salvamento dos Gráficos Profissionais
plotar_e_salvar_graficos(df_rede, hist_vol, hist_pres, hist_efic)
```

```
# Passo EXTRA: Exportação da Base de Engenharia processada para arquivo CSV
arquivo_csv = "relatorio_gas_bairros.csv"
df_rede.to_csv(arquivo_csv, index=False, sep=';', encoding='utf-8-sig')
print(f"[EXTRA] Base de dados gerada foi salva com sucesso no formato padrão:
'{arquivo_csv}'")
print("\n[INFO] Execução do sistema finalizada com sucesso. Pronto para submissão
acadêmica.")
```

```
if __name__ == "__main__":
    main()
```